

TARGET ASSET	AUDIT TOOL	TIMESTAMP
http:// example.com	ZAP	2026-03-18 09:13:24

STRATEGIC OVERVIEW

As NetShieldAI's Senior Web Application Security Consultant, I've analyzed the OWASP ZAP scan results for your web application. Below is a comprehensive summary of the findings, their potential impact, and actionable recommendations for remediation.

1. Executive Summary Table

SCAN TARGET	SCAN DATE	HIGH RISKS	MEDIUM RISKS	LOW/ INFO RISKS	OVERALL STATUS
http:// example.com	2026-03-18	0	1	1	Needs Attention

2. Critical Vulnerability Analysis (High & Medium Only)

Vulnerability Name & Risk Level: Content Security Policy (CSP) Header Not Set - MEDIUM

- **The 'Plain English' Explanation:** Imagine your web browser is visiting a website. Normally, it will load various resources like JavaScript files, CSS stylesheets, images, and fonts from different places (domains). A Content Security Policy (CSP) is like a strict bouncer at a club. It tells the browser exactly *which* sources are allowed to provide content for your website. If the browser tries to

load something from an unapproved source, the CSP tells it to block that content, even if a malicious script tries to sneak it in. This significantly reduces the risk of certain attacks.

- **Business Impact:** Without a CSP, your website is more vulnerable to attacks like Cross-Site Scripting (XSS) and data injection.

- **XSS:** An attacker could inject malicious scripts into your website, which then run in your users' browsers. This could lead to session hijacking (stealing login cookies), defacing your website, redirecting users to malicious sites, or even stealing sensitive user data directly from the browser.
- **Data Injection:** Similar to XSS, attackers could inject harmful content that alters the appearance or behavior of your site, potentially leading to reputational damage, loss of user trust, and even legal liabilities if user data is compromised.

- **Technical Details:**

- **Affected URL:** `http://example.com`
- The web server or application serving `http://example.com` is not sending the `Content-Security-Policy` HTTP header in its responses.

- **Technical Solution:**

To implement a CSP, you need to configure your web server (e.g., Apache, Nginx), application server (e.g., Node.js, Tomcat), or load balancer to include the `Content-Security-Policy` HTTP header in all relevant web page responses.

A basic CSP might look like this:

```
Content-Security-Policy: default-src 'self'; script-src  
'self' https://trusted-cdn.com; style-src 'self'; img-src  
'self' data;;
```

Let's break down this example:

* `default-src 'self'`: This is the fallback policy. It means that by default, all resources (scripts, styles, images, etc.) should only be loaded from the same origin as the document (`'self'`).

* `script-src 'self' https://trusted-cdn.com`: This specifically allows JavaScript to be loaded from your own domain (`'self'`) and from `https://trusted-cdn.com`.

* `style-src 'self'`: CSS stylesheets can only be loaded from your own domain.

* `img-src 'self' data::` Images can be loaded from your own domain or from `data:` URIs (base64 encoded images).

You should carefully design your CSP to only allow resources from trusted sources that your application legitimately needs. Start with a strict policy and gradually relax it as you identify necessary third-party resources. You can also use the `Content-Security-Policy-Report-Only` header to test your policy without enforcing it, allowing you to see violations in your server logs.

3. Prioritized Remediation Checklist

PRIORITY	ACTION ITEM	AFFECTED COMPONENT	DIFFICULTY
1	Implement Content Security Policy (CSP)	Web Server/Application Configuration	Medium
2	Set X-Content-Type-Options Header	Web Server/Application Configuration	Low

4. Low Risk & Best Practices

- **X-Content-Type-Options Header Missing (LOW):**

- **Explanation:** This header protects against MIME-sniffing attacks. Without it, some browsers might try to guess the content type of a file (e.g., treat a `.txt` file as a `.js` file) if the server doesn't explicitly state it. An attacker could potentially upload a malicious file with a misleading extension, and if the browser "sniffs" it as an executable script, it could run.
- **Recommendation:** Configure your web server to include the `X-Content-Type-Options: nosniff` header in all responses. This tells the browser to strictly follow the `Content-Type` header provided by the server and not to perform MIME sniffing. This is a quick and effective defense-in-depth measure.

